

Manual (de)initialization

A Rust and C++ showcase

Aleksander Krauze
github.com/aleksanderkrauze
aleksander.krauze@zoho.com
exein.io

```
1 trait Exein: Rust + Security {}
```

About

Boring code



```
1 pub struct Foo {
2     name: String,
3     data: Vec<i32>,
4     len: usize,
5 }
6
7 impl Foo {
8     pub fn new(name: &str) -> Self {
9         let name = name.to_owned();
10        let len = name.len();
11
12        Self {
13            name,
14            data: Vec::new(),
15            len,
16        }
17    }
18 }
19
20 fn main() {
21     let foo = Foo::new("Rustikon");
22 }
```

```
1 #include <string>
2 #include <vector>
3
4 class Foo {
5     std::string m_name;
6     std::vector<int> m_data;
7     std::size_t m_len;
8
9 public:
10     explicit Foo(const char *name)
11         : m_name{name}
12         , m_data{}
13         , m_len{this->m_name.length()}
14     {}
15 };
16
17 int main() {
18     Foo foo = Foo{"Rustikon"};
19
20     return 0;
21 }
```

Manual memory initialization in Rust

`std::mem::MaybeUninit`



```
1 use std::mem::MaybeUninit;
2 use std::ptr;
3
4 pub struct Foo {
5     name: String,
6     data: Vec<i32>,
7     len: usize,
8 }
9
10 impl Foo {
11     pub fn new_in<'storage>(
12         this: &mut MaybeUninit<Self>,
13         name: &str
14     ) -> &'storage mut Self {
15         let this = this.as_mut_ptr();
16
17         let name = name.to_owned();
18         let data = Vec::new();
19
20         unsafe {
21             ptr::write(&raw mut (*this).name, name);
22             ptr::write(&raw mut (*this).data, data);
23
24             let len = unsafe {
25                 let name = &(&raw const (*this).name);
26                 name.len()
27             };
28             ptr::write(&raw mut (*this).len, len);
29         }
30
31         unsafe { &mut *this }
32     }
33 }
```

```
34 fn use_foo(_foo: &Foo) {}
35
36 fn main() {
37     let mut storage = MaybeUninit::uninit();
38
39     let foo = Foo::new_in(&mut storage, "Rustikon");
40     use_foo(&*foo);
41
42     unsafe {
43         storage.assume_init_drop();
44     }
45 }
```


Manual memory initialization in C++



```
1 #include <string>
2 #include <vector>
3
4 class Foo {
5     std::string m_name;
6     std::vector<int> m_data;
7     std::size_t m_len;
8
9 public:
10     explicit Foo(const char *name)
11         : m_name{name},
12         m_data{},
13         m_len{this->m_name.length()}
14     {}
15 };
16
17 void use_foo(const Foo &foo) {}
```

```
18
19 int main() {
20     alignas(Foo) std::byte buf[sizeof(Foo)];
21
22     Foo *foo = new (buf) Foo{"Rustikon"};
23     use_foo(*foo);
24     foo->~Foo();
25
26     return 0;
27 }
```

Destructors, Drop, and Drop-glue

Normal memory deinitialization in C++



```
1 #include <print>
2 #include <string>
3
4 class Logger {
5     std::string m_message;
6
7 public:
8     Logger(const char *message) :
9         m_message{message} {}
10    ~Logger() { std::println!("{}", m_message); }
11 };
12
13 class Foo {
14     Logger a;
15     Logger b;
16     Logger c;
17
18 public:
19     Foo() : a{"a"}, b{"b"}, c{"c"} {}
20     ~Foo() { std::println("Foo"); }
21 };
22
23 int main() {
24     Foo foo = Foo{};
25
26     return 0;
27 }
```

```
1 Output:
2 Foo
3 c
4 b
5 a
```

Normal memory deinitialization in Rust



```
1 pub struct Logger(String);
2
3 impl Drop for Logger {
4     fn drop(&mut self) {
5         println!("{}", self.0);
6     }
7 }
8
9 pub struct Foo {
10     a: Logger,
11     b: Logger,
12     c: Logger,
13 }
14
15 impl Drop for Foo {
16     fn drop(&mut self) {
17         println!("Foo");
18     }
19 }
```

```
20
21 impl Foo {
22     pub fn new() -> Self {
23         Self {
24             a: Logger("a".to_owned()),
25             b: Logger("b".to_owned()),
26             c: Logger("c".to_owned()),
27         }
28     }
29 }
30
31 fn main() {
32     let foo = Foo::new();
33 }
```

```
1 Output:
2 Foo
3 a
4 b
5 c
```

Controlling drop order

- 1. Disable drop-glue**
- 2. Run destructors manually**

std::mem::ManuallyDrop



```
1 use std::mem::ManuallyDrop;
2
3 pub struct Logger(String);
4
5 impl Drop for Logger {
6     fn drop(&mut self) {
7         println!("{}", self.0);
8     }
9 }
10
11 struct Foo {
12     a: ManuallyDrop<Logger>,
13     b: ManuallyDrop<Logger>,
14     c: ManuallyDrop<Logger>,
15 }
16
17 impl Drop for Foo {
18     fn drop(&mut self) {
19         unsafe {
20             ManuallyDrop::drop(&mut self.a);
21             ManuallyDrop::drop(&mut self.c);
22             ManuallyDrop::drop(&mut self.b);
23         }
24     }
25 }
26
```

```
27 impl Foo {
28     fn new() -> Self {
29         Self {
30             a: ManuallyDrop::new(Logger("a".to_owned())),
31             b: ManuallyDrop::new(Logger("b".to_owned())),
32             c: ManuallyDrop::new(Logger("c".to_owned())),
33         }
34     }
35 }
36
37 fn main() {
38     let foo = Foo::new();
39 }

```

1 Output:

```
2 a
3 c
4 b
```

C++

With runtime overhead:

- `std::optional`
- `std::unique_ptr`

union



```
1 #include <print>
2 #include <string>
3
4 class Logger {
5     std::string m_message;
6
7 public:
8     Logger(const char *message) :
9         m_message{message} {}
10    ~Logger() { std::println("{} ", m_message); }
11 };
12
13 class Foo {
14     union { Logger a; };
15     union { Logger b; };
16     union { Logger c; };
17
18 public:
19     Foo() : a{"a"}, b{"b"}, c{"c"} {}
20     ~Foo() {
21         a.~Logger();
22         c.~Logger();
23         b.~Logger();
24     }
25 };
26
27 }
```

```
26 int main() {
27     Foo foo = Foo{};
28
29     return 0;
30 }
```

```
1 Output:
2 a
3 c
4 b
```

Summary

	Rust	C++
initialization order	unspecified	In the order of declaration
in-place initialization	Manual. Open design space	Yes. Placement new
destruction order	In the order of declaration	In the <i>reverse</i> order of declaration
controlling destruction order	ManuallyDrop	union



Public repository with slides